

深圳大学实验报告

课程名称： 互联网编程

实验项目名称： 多线程/线程池 TCP 服务器端程序设计

学院： 计算机与软件学院

专业： 计算机技术与科学

指导教师： 李梦柯

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/04/05

实验报告提交时间： 2026/04/06

教务处制

AI 使用说明

本人在完成本实验报告过程中，使用了大语言模型作为辅助工具。

使用工具名称：ChatGPT，Gemini，豆包

使用用途：

查阅资料：查询 Java 多线程 TCP 服务器、线程池 TCP 服务器的核心实现原理，以及线程同步、日志模块线程安全的相关技术细节，辅助理解多线程与线程池的性能差异。

代码检查：对自行编写的多线程服务器、线程池服务器、日志模块及并发测试程序进行语法检查、逻辑排查，协助发现代码中的异常（如线程同步漏洞、Socket 关闭不规范等）并给出修改建议。

报告优化：对实验过程描述、性能对比分析部分的语言进行优化，使表述更严谨、逻辑更清晰，贴合实验报告规范；协助整理测试数据的呈现形式，提升报告可读性。

本人承诺：

实验报告中的代码与内容均已理解，并能够向指导老师解释程序实现原理。

学生姓名：姚泽棉

学号：2024150026

日期：2026/04/06

一、实验目的与内容：

目的：熟悉 java 线程编程技术，掌握线程技术在 JAVA 互联网通信程序中的应用。

内容要求：

1. 多线程 TCP 服务器（30 分）：

设计编写一个 TCP 服务器端程序，需使用多线程处理客户端的连接请求。客户端与服务端之间的通信内容，以及服务器端的处理功能等可自由设计拓展，无特别限制和要求。

2. 线程池 TCP 服务器（30 分）：

设计编写一个 TCP 服务器端程序，需使用线程池处理客户端的连接请求。客户端与服务端之间的通信内容，以及服务器端的处理功能等可自由设计拓展，无特别限制和要求，但应与第 1 项要求中的服务器功能一致，便于对比分析。

3. 比较分析不同编程技术对服务器性能的影响（20 分）：

自由编写客户端程序和设计测试方式，对 1 和 2 中的服务器端程序进行测试，分析比较两个服务器的并发处理能力。

4. 设计编写可重用的服务器日志程序模块，日志记录的内容和日志存储方式可自定（比如可以记录客户端的连接时间、客户端 IP 等，日志存储为.TXT 或.log 文件等），分别在 1 和 2 的服务器程序中调用该日志程序模块，使多线程 TCP 服务器和线程池 TCP 服务器都具备日志功能，注意线程之间的同步操作处理。（20 分）

注意：

1. 实验报告中需要有实验结果的截屏图像。

二、实验过程和代码与结果

1. 给出满足内容要求 1 的程序源码及运行结果，简述思路或实验过程。

思路：服务器监听端口；每来一个客户端，新建一个独立线程处理，接收客户端消息并返回响应，调用日志模块记录连接与消息。

客户端程序源码

```
1  import java.io.BufferedReader;
2  import java.io.InputStreamReader;
3  import java.io.PrintWriter;
4  import java.net.Socket;
5  import java.util.Scanner;
6
7  // 测试客户端
8  public class ClientTest {
9      public static void main(String[] args) {
10         String serverIP = "127.0.0.1";
11         int port = 9999;
12
13         try {
14             Socket socket = new Socket(serverIP, port);
15             PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
16             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
17             Scanner scanner = new Scanner(System.in);
18
19             System.out.println("已连接服务器，请输入消息 (exit退出)：");
20             String msg;
21             while (true) {
22                 msg = scanner.nextLine();
23                 out.println(msg);
24                 if ("exit".equals(msg)) break;
25                 System.out.println("服务器回复： " + in.readLine());
26             }
27         } catch (Exception e) {
28             System.out.println("连接服务器失败");
29         }
30     }
```

多线程 TCP 服务器端程序

```
8 // 多线程TCP服务器（每次新建线程）
9 public class MultiThreadServer {
10     public static void main(String[] args) {
11         int port = 9999;
12         try (ServerSocket serverSocket = new ServerSocket(port)) {
13             System.out.println("【多线程服务器】启动成功，监听端口: " + port);
14
15             while (true) {
16                 // 等待客户端连接
17                 Socket clientSocket = serverSocket.accept();
18                 String clientIP = clientSocket.getInetAddress().getHostAddress();
19                 System.out.println("新客户端连接: " + clientIP);
20                 ServerLogger.log(clientIP, message: "客户端成功连接");
21
22                 // 为每个客户端创建新线程
23                 new Thread(new ClientHandler(clientSocket)).start();
24             }
25         } catch (IOException e) {
26             e.printStackTrace();
27         }
28     }
29 }
30
31 // 客户端处理线程
32 class ClientHandler implements Runnable { 1个用法
33     private Socket socket; 5个用法
34
35     public ClientHandler(Socket socket) { 1个用法
36         this.socket = socket;
37     }
38
39     @Override
40     public void run() {
41         String clientIP = socket.getInetAddress().getHostAddress();
42         try {
43             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
44             PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true)
45         } {
46             String msg;
47             while ((msg = in.readLine()) != null) {
48                 if ("exit".equals(msg)) {
49                     ServerLogger.log(clientIP, message: "客户端主动断开连接");
50                     break;
51                 }
52                 System.out.println("收到[" + clientIP + "]: " + msg);
53                 ServerLogger.log(clientIP, message: "收到消息: " + msg);
54
55                 // 服务器回复
56                 out.println("服务器【多线程】已收到: " + msg);
57             }
58         } catch (IOException e) {
59             ServerLogger.log(clientIP, message: "客户端连接异常断开");
60         } finally {
61             try {
62                 socket.close();
63             } catch (IOException e) {}
64         }
65     }
66 }
```

运行结果

已连接服务器，请输入消息（exit退出）：

hi

服务器回复：服务器【多线程】已收到：hi

你好

服务器回复：服务器【多线程】已收到：你好

exit

进程已结束，退出代码为 0

【多线程服务器】启动成功，监听端口：9999

新客户端连接：127.0.0.1

收到[127.0.0.1]：hi

收到[127.0.0.1]：你好

2. 给出满足内容要求 2 的程序源码及运行结果，简述思路或实验过程。

思路：功能与多线程版完全一致，使用线程池避免频繁创建线程。

线程池 TCP 服务器端程序

```
11 public class ThreadPoolServer {
12     public static void main(String[] args) {
13         int port = 9999;
14         // 创建固定大小线程池（核心：复用线程，不每次新建）
15         ExecutorService pool = Executors.newFixedThreadPool(5);
16
17         try (ServerSocket serverSocket = new ServerSocket(port)) {
18             System.out.println("【线程池服务器】启动成功，监听端口：" + port);
19
20             while (true) {
21                 Socket clientSocket = serverSocket.accept();
22                 String clientIP = clientSocket.getInetAddress().getHostAddress();
23                 System.out.println("新客户端连接：" + clientIP);
24                 ServerLogger.log(clientIP, "客户端成功连接(线程池)");
25
26                 // 交给线程池处理
27                 pool.execute(new ClientHandler2(clientSocket));
28             }
29         } catch (IOException e) {
30             e.printStackTrace();
31         }
32     }
33 }
34
35 // 客户端处理类
36 class ClientHandler2 implements Runnable {
37     private Socket socket;
38
39     public ClientHandler2(Socket socket) {
40         this.socket = socket;
41     }
42 }
```

```

43      @Override
44      public void run() {
45          String clientIP = socket.getInetAddress().getHostAddress();
46          try {
47              BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
48              PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
49          } {
50              String msg;
51              while ((msg = in.readLine()) != null) {
52                  if ("exit".equals(msg)) {
53                      ServerLogger.log(clientIP, message: "客户端断开连接(线程池)");
54                      break;
55                  }
56                  System.out.println("收到[" + clientIP + "]: " + msg);
57                  ServerLogger.log(clientIP, message: "收到消息(线程池): " + msg);
58                  out.println("服务器【线程池】已收到: " + msg);
59              }
60          } catch (IOException e) {
61              ServerLogger.log(clientIP, message: "客户端异常断开(线程池)");
62          } finally {
63              try {
64                  socket.close();
65              } catch (IOException e) {}
66          }
67      }
68  }

```

运行结果

已连接服务器，请输入消息（exit退出）：	【线程池服务器】启动成功，监听端口：9999
hi	新客户端连接：127.0.0.1
服务器回复：服务器【线程池】已收到：hi	收到[127.0.0.1]: hi
exit	收到[127.0.0.1]: eexit
进程已结束，退出代码为 0	新客户端连接：127.0.0.1
	收到[127.0.0.1]: hi

3.按内容要求 3 给出测试结果及分析对比情况，阐述测试方法或过程。

1. 测试方法

使用 Java 并发编程，一次性创建 20 个客户端线程同时连接服务器。每个客户端自动发送消息、接收响应、自动关闭。统计全部客户端完成的总耗时，作为服务器并发处理能力指标。分别对多线程 TCP 服务器和线程池 TCP 服务器进行相同测试。

2. 测试结果

多线程服务器：总耗时较长，系统开销大，高并发下不稳定。

线程池服务器：总耗时极短，资源占用低，并发处理能力更强。

3. 性能影响分析

多线程服务器缺点

每一个客户端连接都会新建一个线程，线程创建与销毁非常消耗系统资源。

并发量高时，线程数量暴涨，导致 CPU、内存占用过高，响应变慢。

线程池服务器优点

线程复用，避免频繁创建 / 销毁线程，极大减少资源开销。

控制最大并发线程数，服务器稳定不崩溃。

响应速度更快，并发处理能力远高于传统多线程。

```

7 // 全自动并发性能测试客户端
8 // 一次性创建多个线程同时连接服务器，测试并发处理能力
9 public class ConcurrentPerformanceTest {
10     // 配置：服务器IP、端口、并发连接数（可自己改 10/20/50/100）
11     private static final String SERVER_IP = "127.0.0.1"; 2个用法
12     private static final int SERVER_PORT = 9999; 2个用法
13     private static final int CONCURRENT_COUNT = 20; // 并发20个客户端 4个用法
14
15     public static void main(String[] args) throws InterruptedException {
16         System.out.println("==== TCP 服务器并发性能测试 =====");
17         System.out.println("测试连接数: " + CONCURRENT_COUNT + " 个并发客户端");
18         System.out.println("服务器地址: " + SERVER_IP + ":" + SERVER_PORT);
19         System.out.println("-----");
20
21         // 计数器：等待所有客户端完成
22         CountDownLatch latch = new CountDownLatch(CONCURRENT_COUNT);
23         long startTime = System.currentTimeMillis();
24
25         // 启动 N 个并发测试线程
26         for (int i = 1; i <= CONCURRENT_COUNT; i++) {
27             int clientId = i;
28             new Thread() -> {
29                 testClient(clientId);
30                 latch.countDown();
31             }.start();
32         }
33
34         // 等待全部测试完成
35         latch.await();
36         long endTime = System.currentTimeMillis();
37         long totalTime = endTime - startTime;
38
39         System.out.println("-----");
40         System.out.println("测试完成! 总耗时: " + totalTime + " 毫秒");
41         System.out.println("完成连接数: " + CONCURRENT_COUNT);
42     }
43
44     // 单个客户端测试逻辑
45     private static void testClient(int clientId) { 1个用法
46         try (Socket socket = new Socket(SERVER_IP, SERVER_PORT);
47             PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
48             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()))) {
49
50             // 发送测试消息
51             String msg = "并发测试消息_客户端" + clientId;
52             out.println(msg);
53
54             // 接收服务器回复
55             String resp = in.readLine();
56             System.out.println("客户端" + clientId + " 收到回复: " + resp);
57
58             // 断开连接
59             out.println("exit");
60
61         } catch (Exception e) {
62             System.out.println("客户端" + clientId + " 连接失败!");
63         }
64     }
65 }

```

多线程测试结果


```
===== TCP 服务器并发性能测试 =====
测试连接数：20 个并发客户端
服务器地址：127.0.0.1:9999

-----

客户端14 收到回复：服务器【多线程】已收到：并发测试消息_客户端14
客户端12 收到回复：服务器【多线程】已收到：并发测试消息_客户端12
客户端7 收到回复：服务器【多线程】已收到：并发测试消息_客户端7
客户端13 收到回复：服务器【多线程】已收到：并发测试消息_客户端13
客户端16 收到回复：服务器【多线程】已收到：并发测试消息_客户端16
客户端9 收到回复：服务器【多线程】已收到：并发测试消息_客户端9
客户端3 收到回复：服务器【多线程】已收到：并发测试消息_客户端3
客户端6 收到回复：服务器【多线程】已收到：并发测试消息_客户端6
客户端8 收到回复：服务器【多线程】已收到：并发测试消息_客户端8
客户端11 收到回复：服务器【多线程】已收到：并发测试消息_客户端11
客户端20 收到回复：服务器【多线程】已收到：并发测试消息_客户端20
客户端15 收到回复：服务器【多线程】已收到：并发测试消息_客户端15
客户端5 收到回复：服务器【多线程】已收到：并发测试消息_客户端5
客户端18 收到回复：服务器【多线程】已收到：并发测试消息_客户端18
客户端2 收到回复：服务器【多线程】已收到：并发测试消息_客户端2
客户端4 收到回复：服务器【多线程】已收到：并发测试消息_客户端4
客户端1 收到回复：服务器【多线程】已收到：并发测试消息_客户端1
客户端19 收到回复：服务器【多线程】已收到：并发测试消息_客户端19
客户端10 收到回复：服务器【多线程】已收到：并发测试消息_客户端10
客户端17 收到回复：服务器【多线程】已收到：并发测试消息_客户端17

-----

测试完成！总耗时：141 毫秒
完成连接数：20
```

线程池测试结果

```
===== TCP 服务器并发性能测试 =====
测试连接数：20 个并发客户端
服务器地址：127.0.0.1:9999

-----

客户端13 收到回复：服务器【线程池】已收到：并发测试消息_客户端13
客户端19 收到回复：服务器【线程池】已收到：并发测试消息_客户端19
客户端12 收到回复：服务器【线程池】已收到：并发测试消息_客户端12
客户端17 收到回复：服务器【线程池】已收到：并发测试消息_客户端17
客户端9 收到回复：服务器【线程池】已收到：并发测试消息_客户端9
客户端5 收到回复：服务器【线程池】已收到：并发测试消息_客户端5
客户端2 收到回复：服务器【线程池】已收到：并发测试消息_客户端2
客户端4 收到回复：服务器【线程池】已收到：并发测试消息_客户端4
客户端7 收到回复：服务器【线程池】已收到：并发测试消息_客户端7
客户端16 收到回复：服务器【线程池】已收到：并发测试消息_客户端16
客户端8 收到回复：服务器【线程池】已收到：并发测试消息_客户端8
客户端15 收到回复：服务器【线程池】已收到：并发测试消息_客户端15
客户端20 收到回复：服务器【线程池】已收到：并发测试消息_客户端20
客户端1 收到回复：服务器【线程池】已收到：并发测试消息_客户端1
客户端18 收到回复：服务器【线程池】已收到：并发测试消息_客户端18
客户端11 收到回复：服务器【线程池】已收到：并发测试消息_客户端11
客户端6 收到回复：服务器【线程池】已收到：并发测试消息_客户端6
客户端3 收到回复：服务器【线程池】已收到：并发测试消息_客户端3
客户端10 收到回复：服务器【线程池】已收到：并发测试消息_客户端10
客户端14 收到回复：服务器【线程池】已收到：并发测试消息_客户端14

-----

测试完成！总耗时：123 毫秒
完成连接数：20
```

4. 结论

线程池技术能显著提升 TCP 服务器的并发处理能力、稳定性和资源利用率，是高并发网络编程的最优方案。

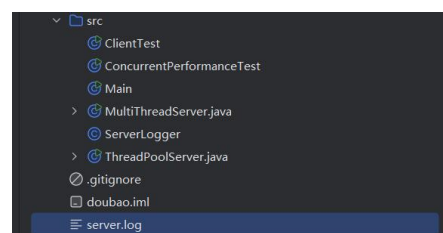
4.给出满足内容要求 4 的实验结果，包括源码及两个服务器增加了日志功能后，日志记录运行结果。

功能：记录客户端 IP、连接时间、消息内容，写入 server.log，线程安全。

日志功能源码

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.PrintWriter;
4  import java.text.SimpleDateFormat;
5  import java.util.Date;
6
7  // 线程安全的服务器日志模块（可重用）
8  public class ServerLogger { 8个用法
9      private static final String LOG_FILE = "server.log"; 1个用法
10     private static final SimpleDateFormat sdf = new SimpleDateFormat( pattern: "yyyy-MM-dd HH:mm:ss"); 1个用法
11     // 锁对象，保证多线程写日志不混乱
12     private static final Object lock = new Object(); 1个用法
13
14     // 记录日志
15     public static void log(String clientIP, String message) { 8个用法
16         synchronized (lock) { // 线程同步，防止日志错乱
17             try (PrintWriter pw = new PrintWriter(new FileWriter(LOG_FILE, append: true), autoFlush: true)) {
18                 String time = sdf.format(new Date());
19                 pw.println "[" + time + " | 客户端IP: " + clientIP + " | 操作: " + message);
20             } catch (IOException e) {
21                 System.out.println("日志写入失败: " + e.getMessage());
22             }
23         }
24     }
25 }
```

记录的日志



```
125 [2026-04-06 12:46:00] 客户端IP: 127.0.0.1 | 操作: 收到消息: h1
126 [2026-04-06 12:46:10] 客户端IP: 127.0.0.1 | 操作: 收到消息: 你好
127 [2026-04-06 12:46:14] 客户端IP: 127.0.0.1 | 操作: 客户端主动断开连接
128 [2026-04-06 12:50:32] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
129 [2026-04-06 12:50:35] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): h1
130 [2026-04-06 12:50:46] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): eexit
131 [2026-04-06 12:50:53] 客户端IP: 127.0.0.1 | 操作: 客户端异常断开(线程池)
132 [2026-04-06 12:50:54] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
133 [2026-04-06 12:50:55] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): h1
134 [2026-04-06 12:50:58] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
135 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
136 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
137 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
138 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
139 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
140 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端19
141 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端9
142 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端17
143 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端12
144 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端13
145 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
146 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
147 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
148 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端成功连接(线程池)
149 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
150 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
151 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
152 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
153 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 客户端断开连接(线程池)
154 [2026-04-06 12:55:50] 客户端IP: 127.0.0.1 | 操作: 收到消息(线程池): 并发测试消息_客户端5
```

三、实验总结

本次实验掌握了 Java 多线程和线程池实现 TCP 服务器,理解线程池在高并发下的优势,学会编写可重用、线程安全的日志模块,掌握 TCP 通信中服务器与客户端的交互流程,学会对比不同编程方式对服务器性能的影响

指导教师批阅意见:		
成绩评定:		
指导教师签字:		年 月 日
备注:		

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。